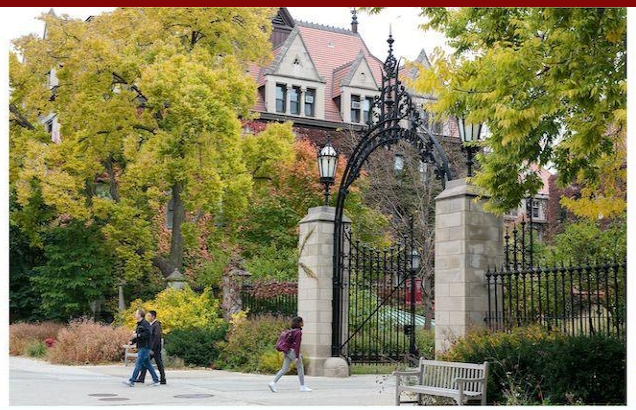
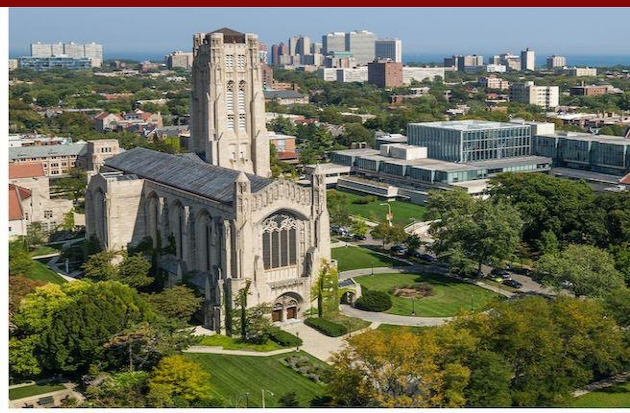
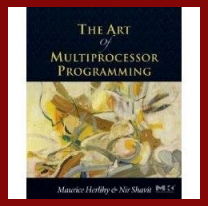
MPCS 52060 - Parallel Programming
M5: Hashtables, Scheduling, Patterns in Go

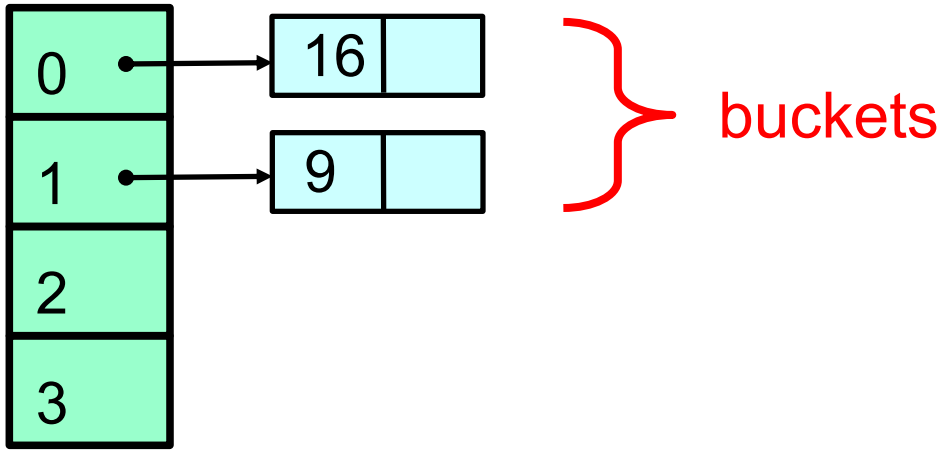


Original slides from “The Art of Multiprocessor Programming by Maurice Herlihy & Nir Shavit” With modifications by Lamont Samuels and Jan Hückelheim

Nonblocking queue demo

Hash tables

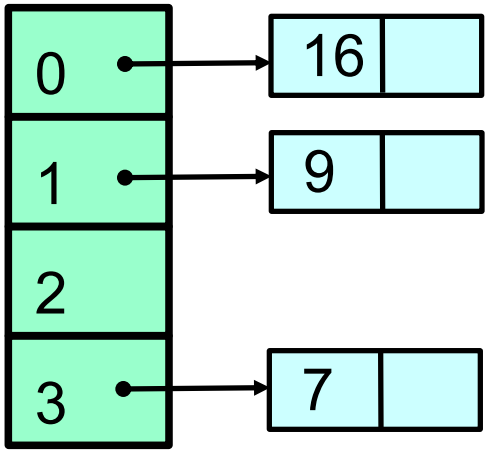
Sequential Closed Hash Map



2 Items

h(k) = k mod 4

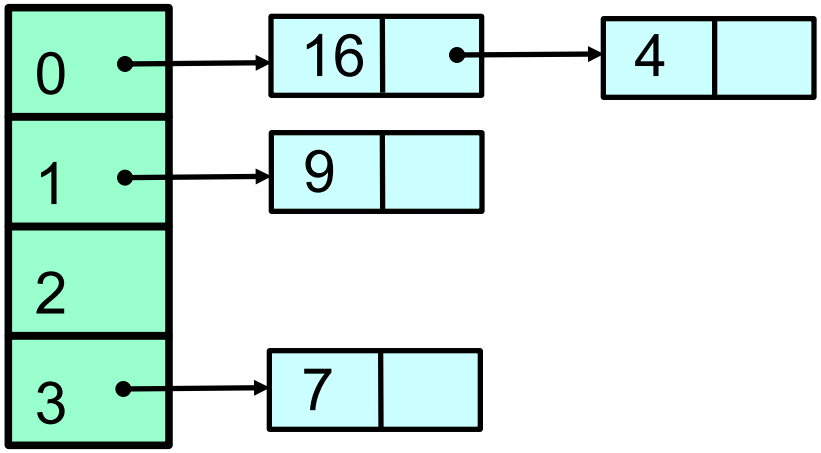
Add an Item



3 Items

h(k) = k mod 4

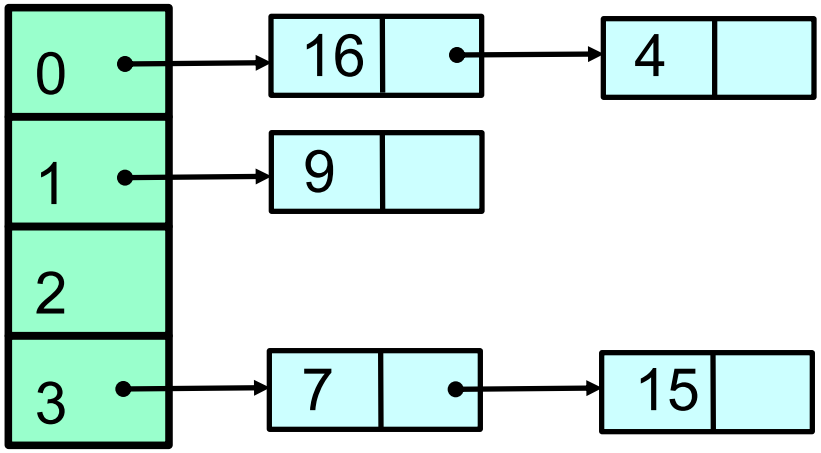
Add Another: Collision



4 Items

h(k) = k mod 4

More Collisions



5 Items

h(k) = k mod 4

Fields

```
public class SimpleHashSet {
  protected LockFreeList[] table;

  public SimpleHashSet(int capacity) {
    table = new LockFreeList[capacity];
    for (int i = 0; i < capacity; i++)
      table[i] = new LockFreeList();
  }
...
```

Array of lock-free lists

Constructor

```
public class SimpleHashSet {
  protected LockFreeList[] table;

  public SimpleHashSet(int capacity) {
    table = new LockFreeList[capacity];
    for (int i = 0; i < capacity; i++)
      table[i] = new LockFreeList();
  }
...
```

Initial size

Constructor

```
public class SimpleHashSet {
  protected LockFreeList[] table;

  public SimpleHashSet(int capacity) {
    table = new LockFreeList[capacity];
    for (int i = 0; i < capacity; i++)
      table[i] = new LockFreeList();
  }
...
```

Allocate memory

Constructor

```
public class SimpleHashSet {
  protected LockFreeList[] table;

  public SimpleHashSet(int capacity) {
    table = new LockFreeList[capacity];
    for (int i = 0; i < capacity; i++)
      table[i] = new LockFreeList();
  }
...
```

Initialization

Add Method

```
public boolean add(Object key) {
 int hash =
  key.hashCode() % table.length;
 return table[hash].add(key);
}
```

Add Method

```
public boolean add(Object key) {
 int hash =
  key.hashCode() % table.length;
 return table[hash].add(key);
}
```

Use object hash code to pick a bucket

Add Method

```
public boolean add(Object key) {
 int hash =
  key.hashCode() % table.length;
 return table[hash].add(key);
}
```

Call bucket's add() method

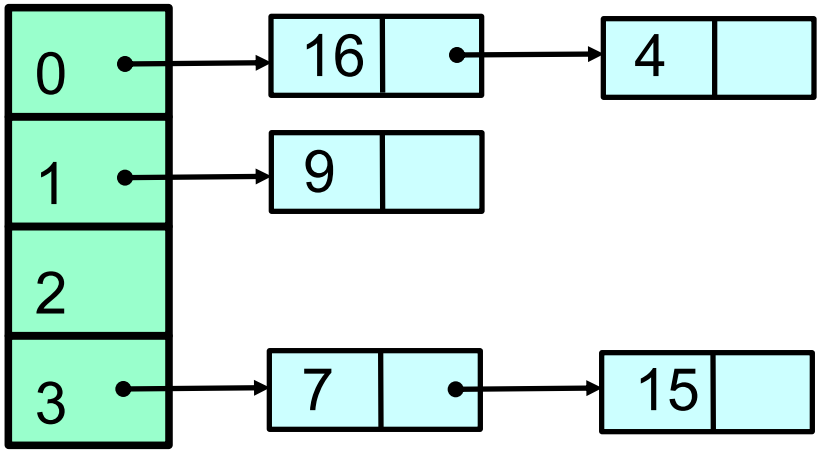
No Brainer?

- We just saw a
  - Simple
  - Lock-free
  - Concurrent hash-based set implementation
- What's not to like?

No Brainer?

- We just saw a
  - Simple
  - Lock-free
  - Concurrent hash-based set implementation
- What's not to like?
- We don't know how to resize …

More Collisions

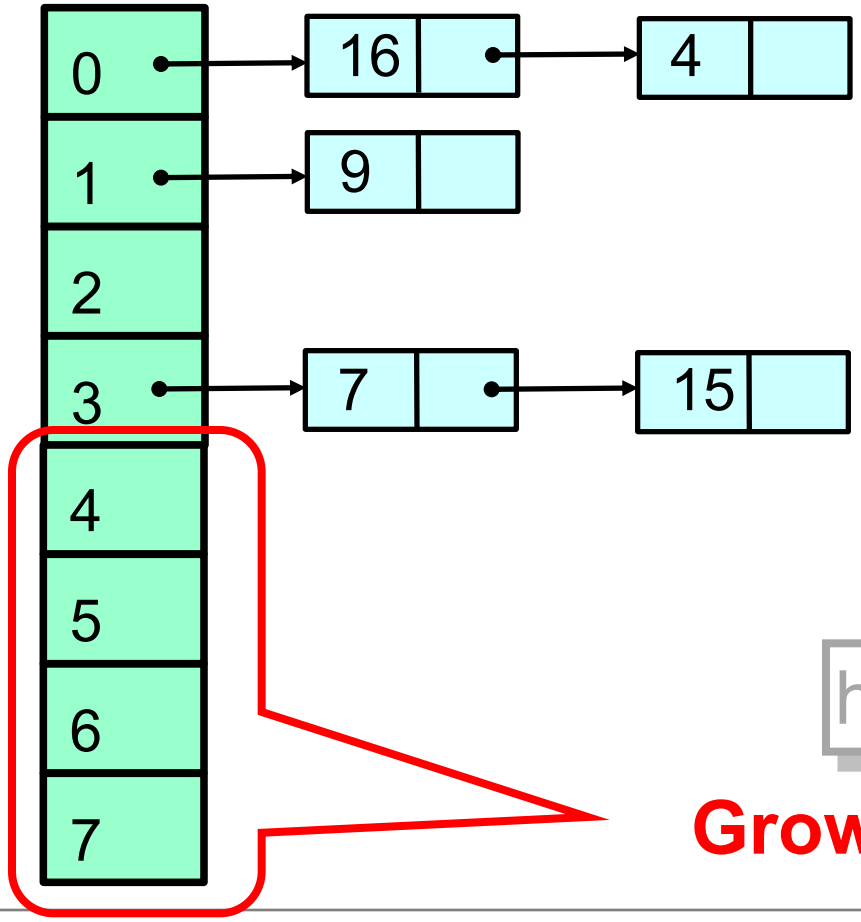


5 Items

Problem:
buckets becoming too long

h(k) = k mod 4

Resizing

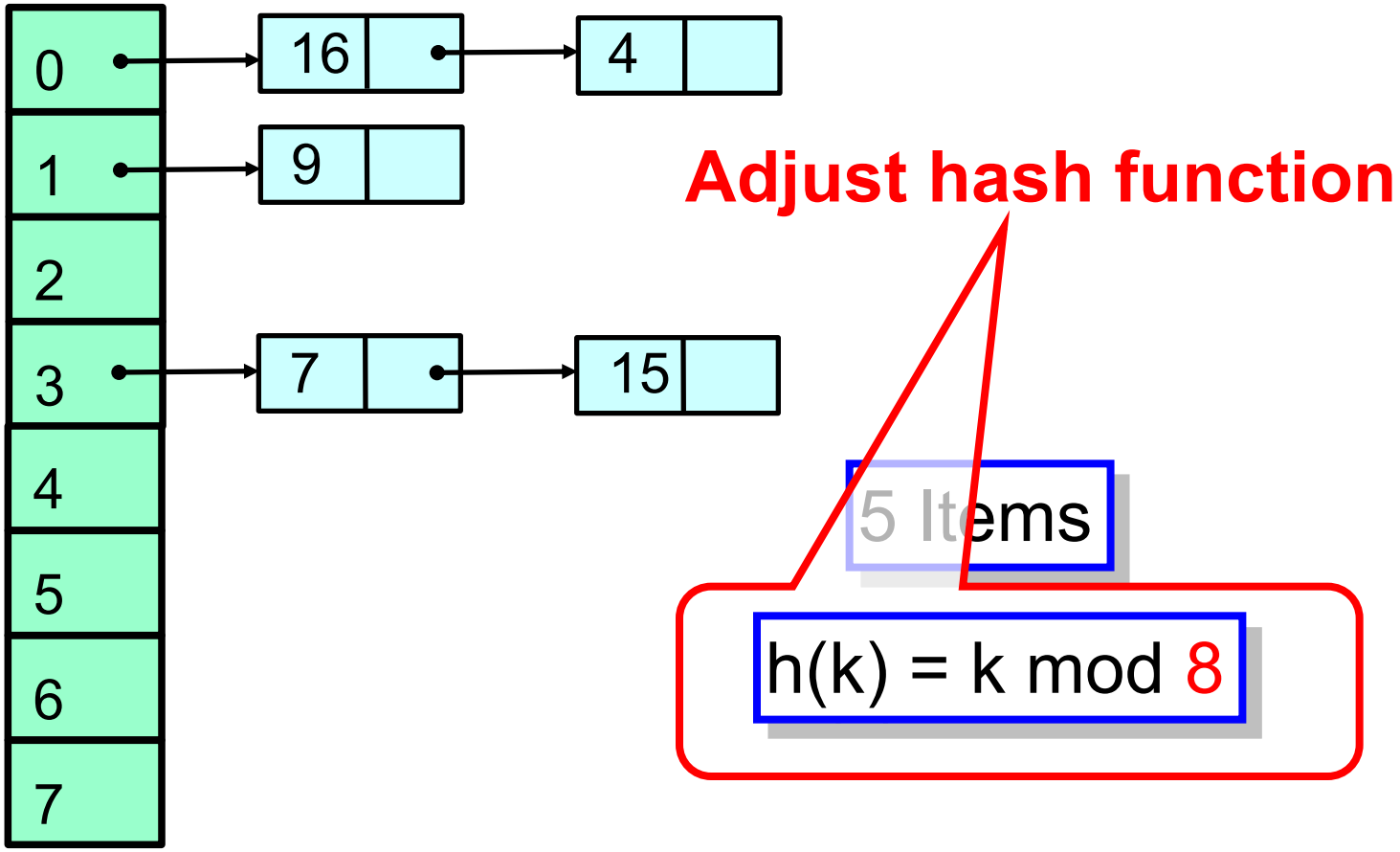


5 Items

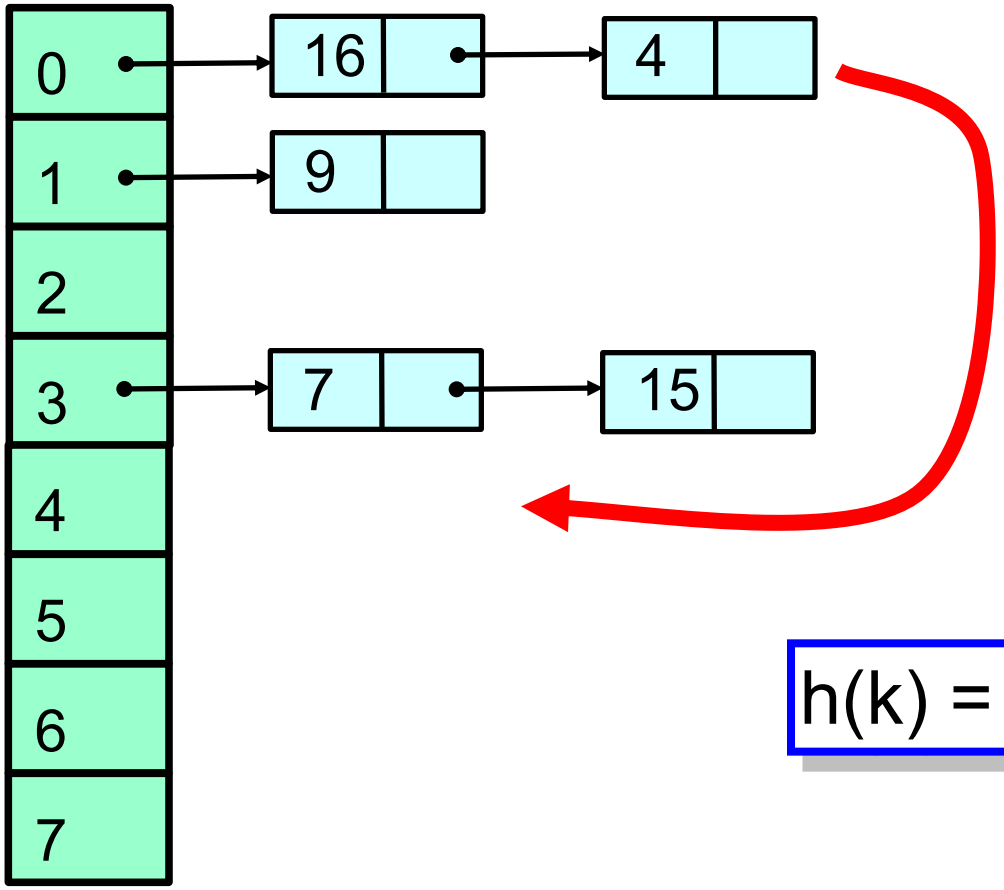
h(k) = k mod 4

Grow the array

Resizing



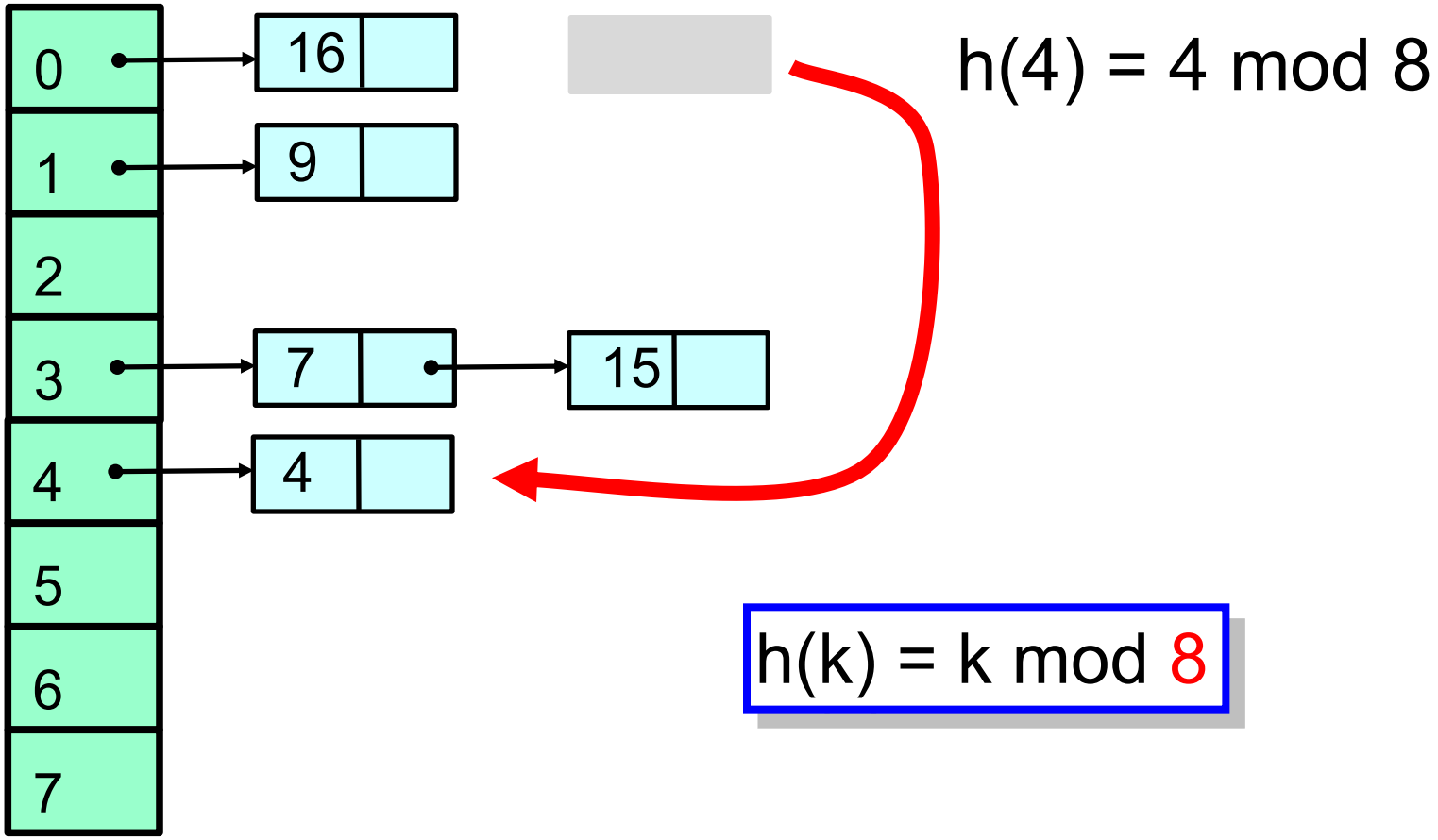
Resizing



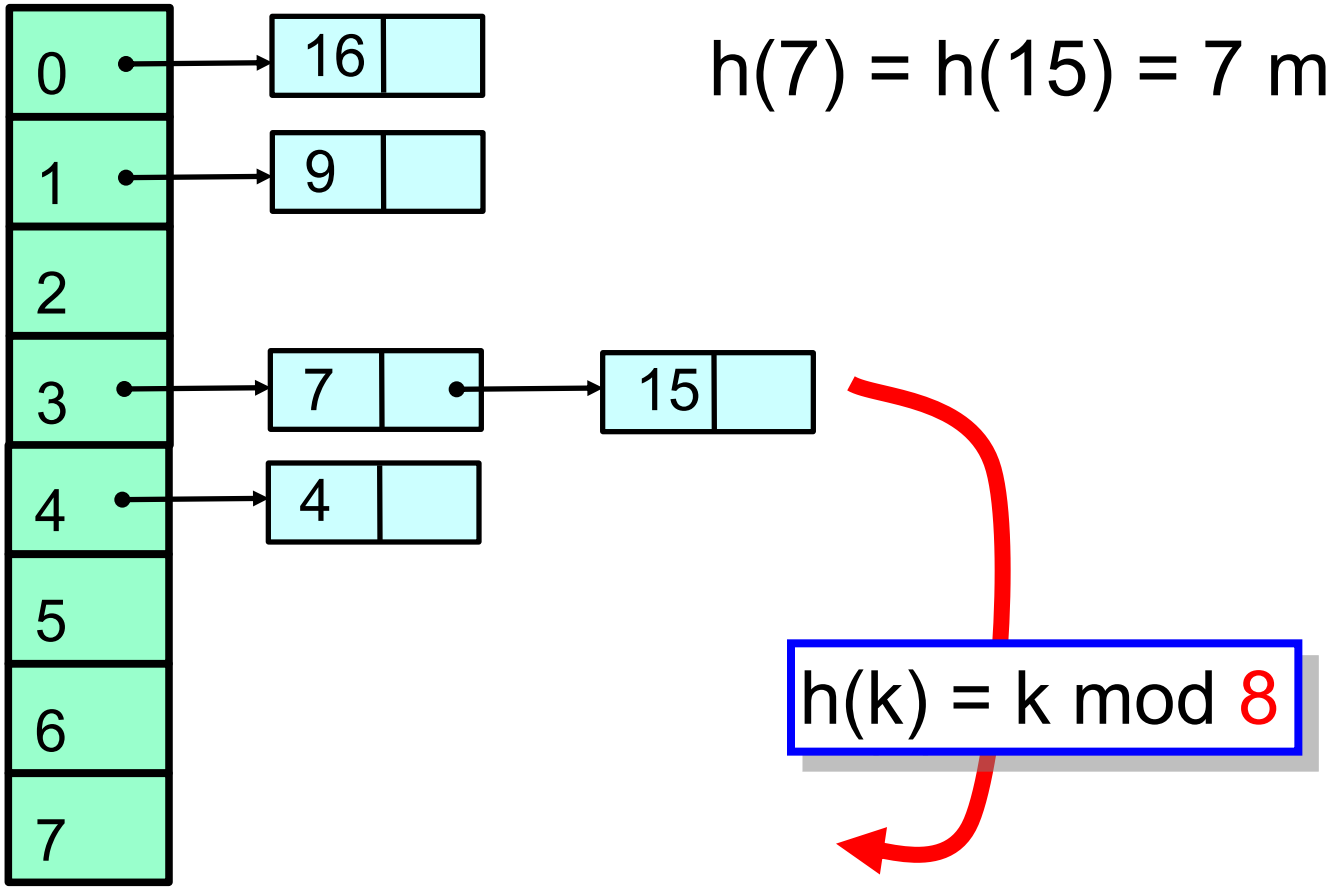
h(4) = 4 mod 8

h(k) = k mod 8

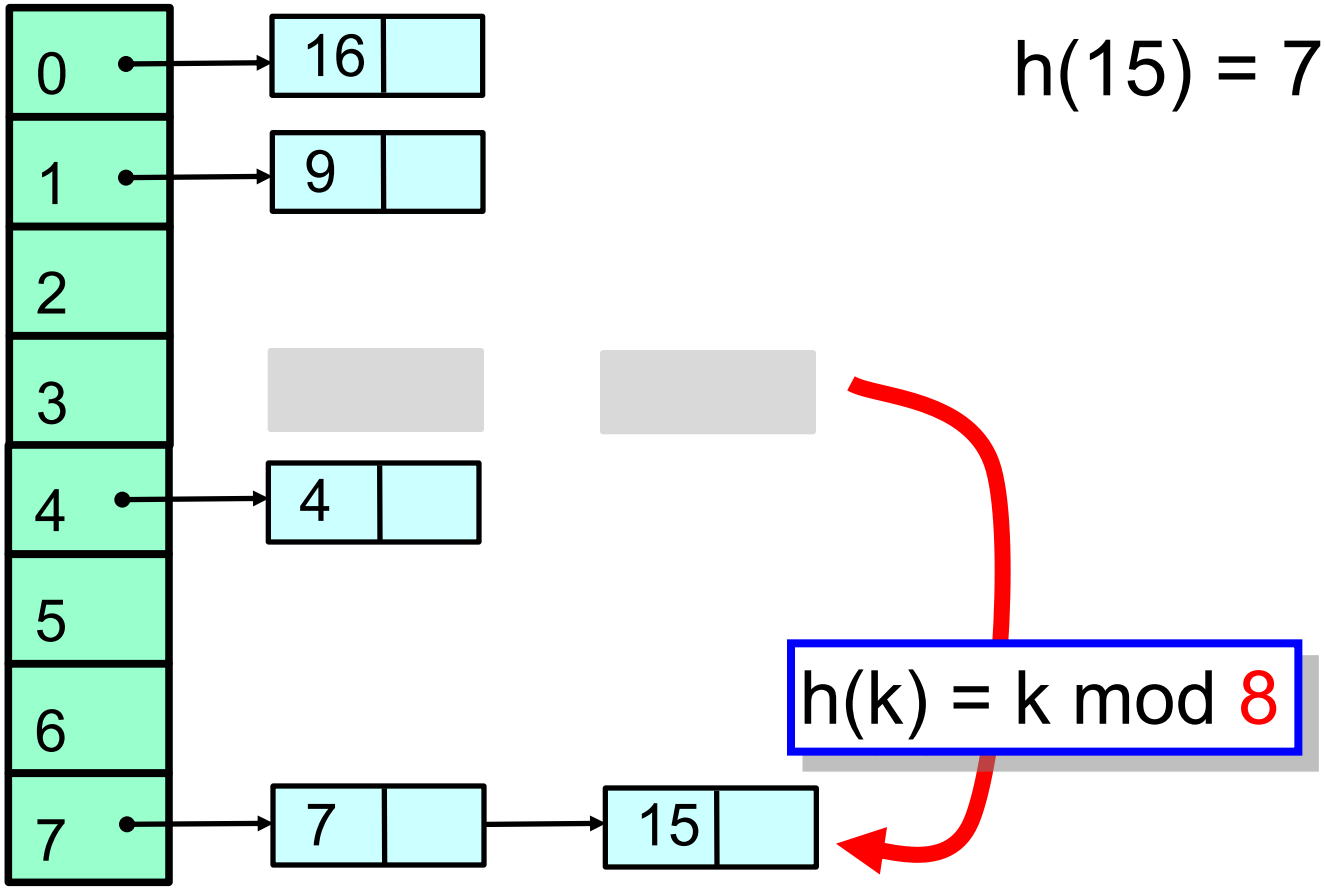
Resizing



Resizing



Resizing



Is Resizing Necessary?

- Constant-time method calls require
  - Constant-length buckets
  - Table size proportional to set size
  - As set grows, must be able to resize

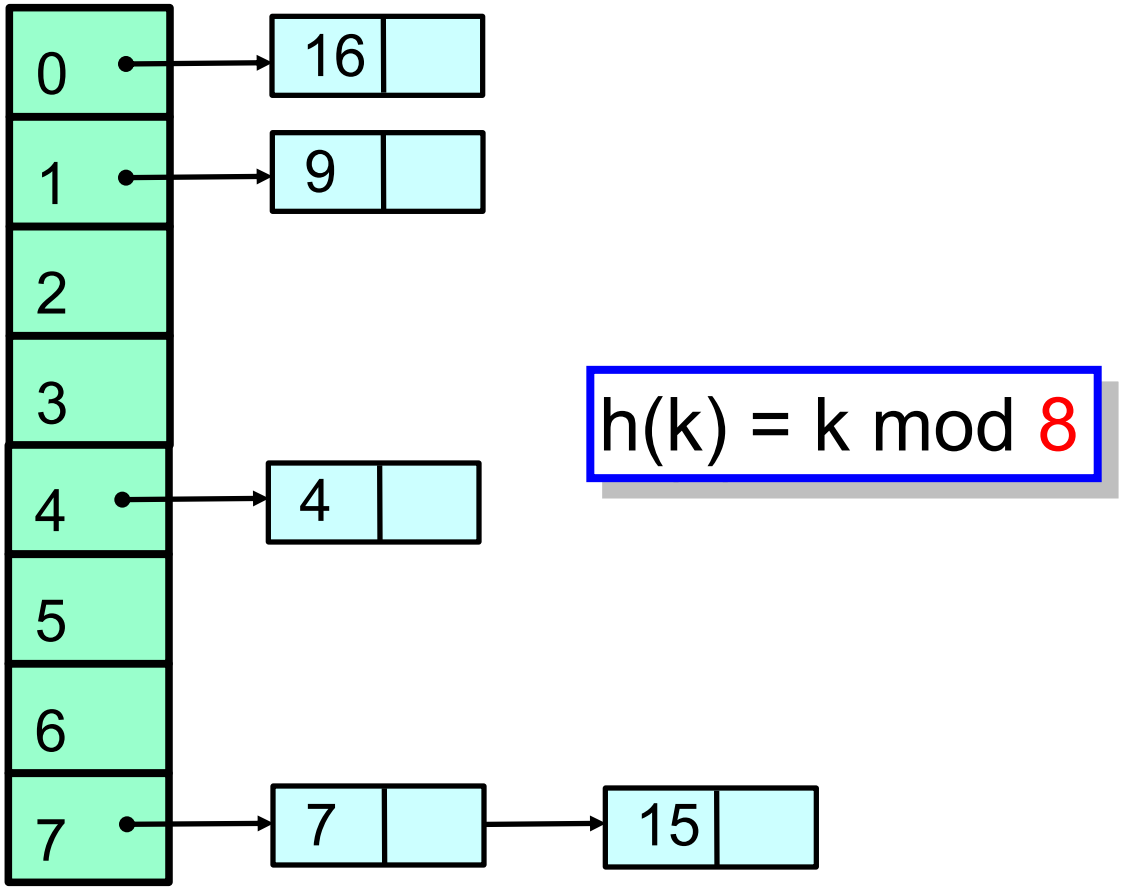
Set Method Mix

- Typical load
  - **90%** contains()
  - **9%** add ()
  - **1%** remove()
- Growing is important
- Shrinking not so much

When to Resize?

- Many reasonable policies. Here's one.
- Pick a threshold on num of items in a bucket
- Global threshold
  - When ≥ ¼ buckets exceed this value
- Bucket threshold
  - When any bucket exceeds this value

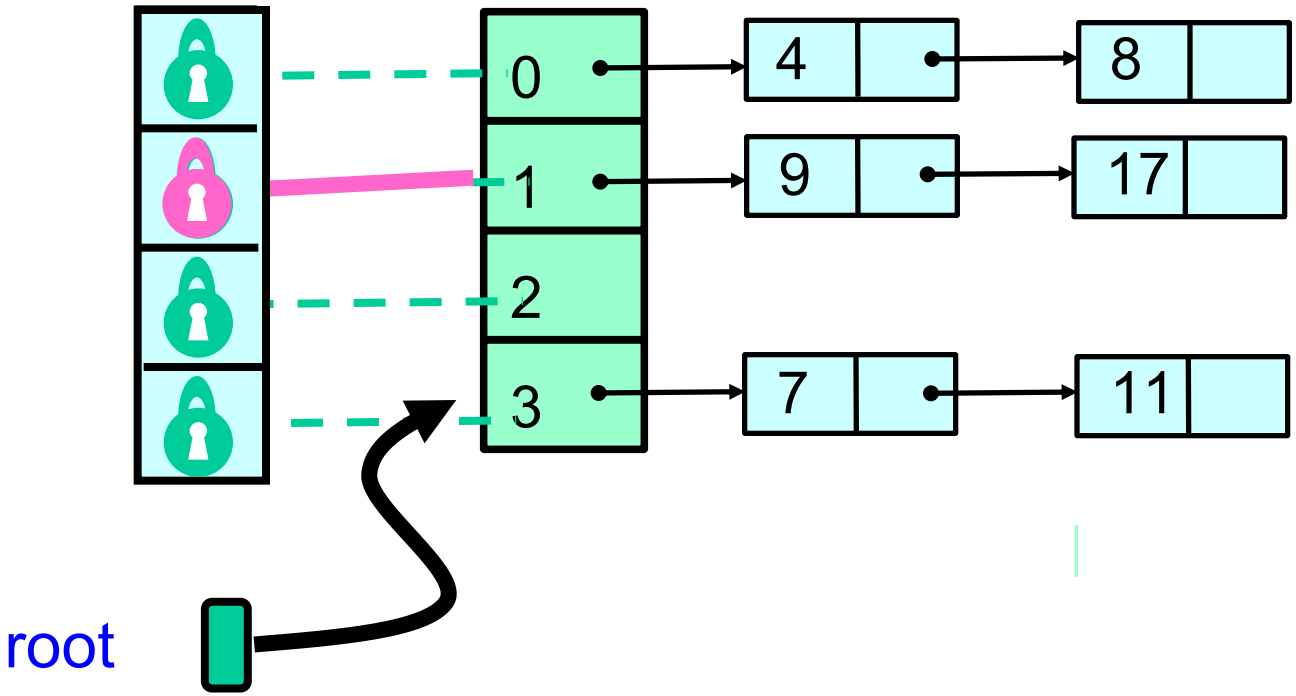
The problem with resizing



Coarse-Grained Locking

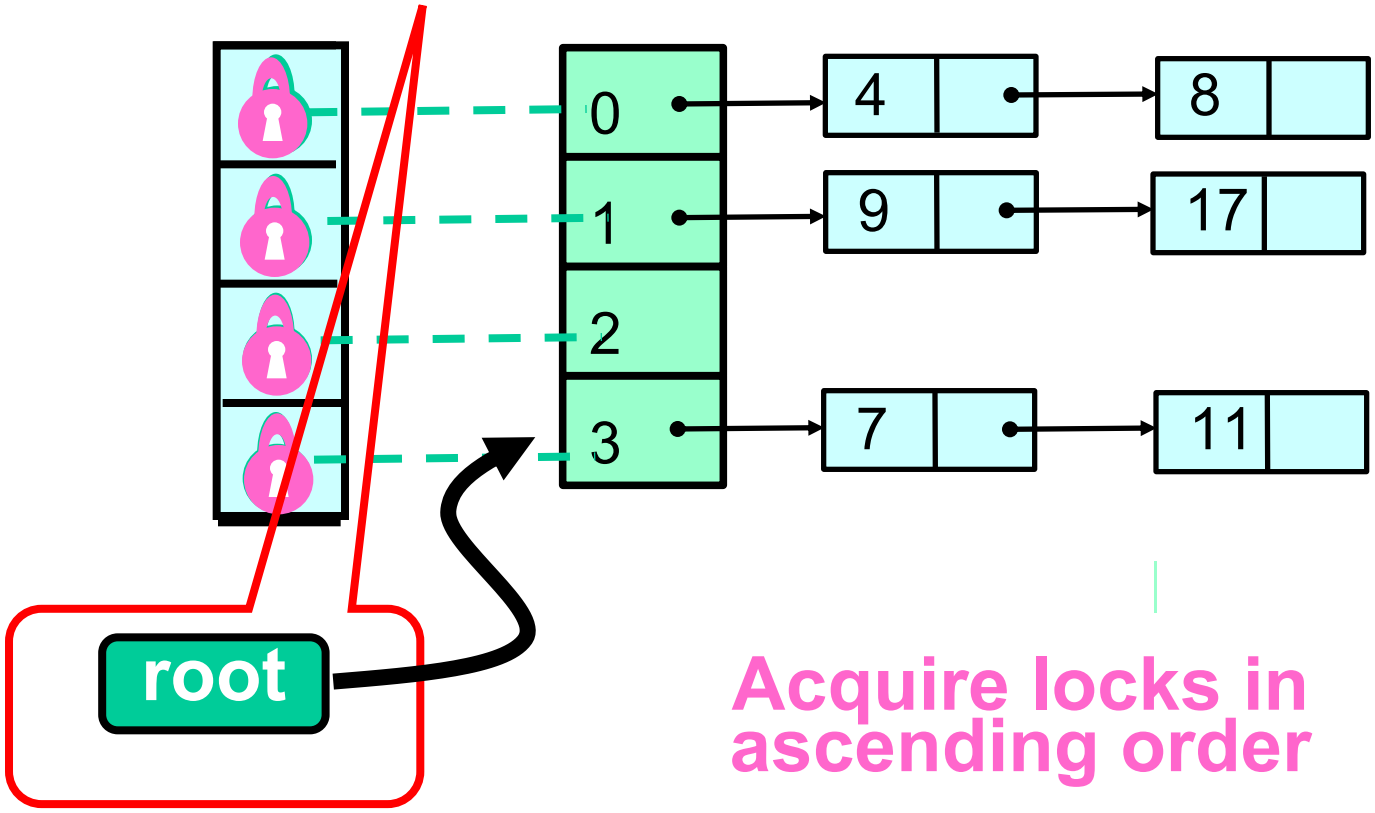
- Good parts
  - Simple
  - Hard to mess up
- Bad parts
  - Sequential bottleneck

Fine-grained Locking

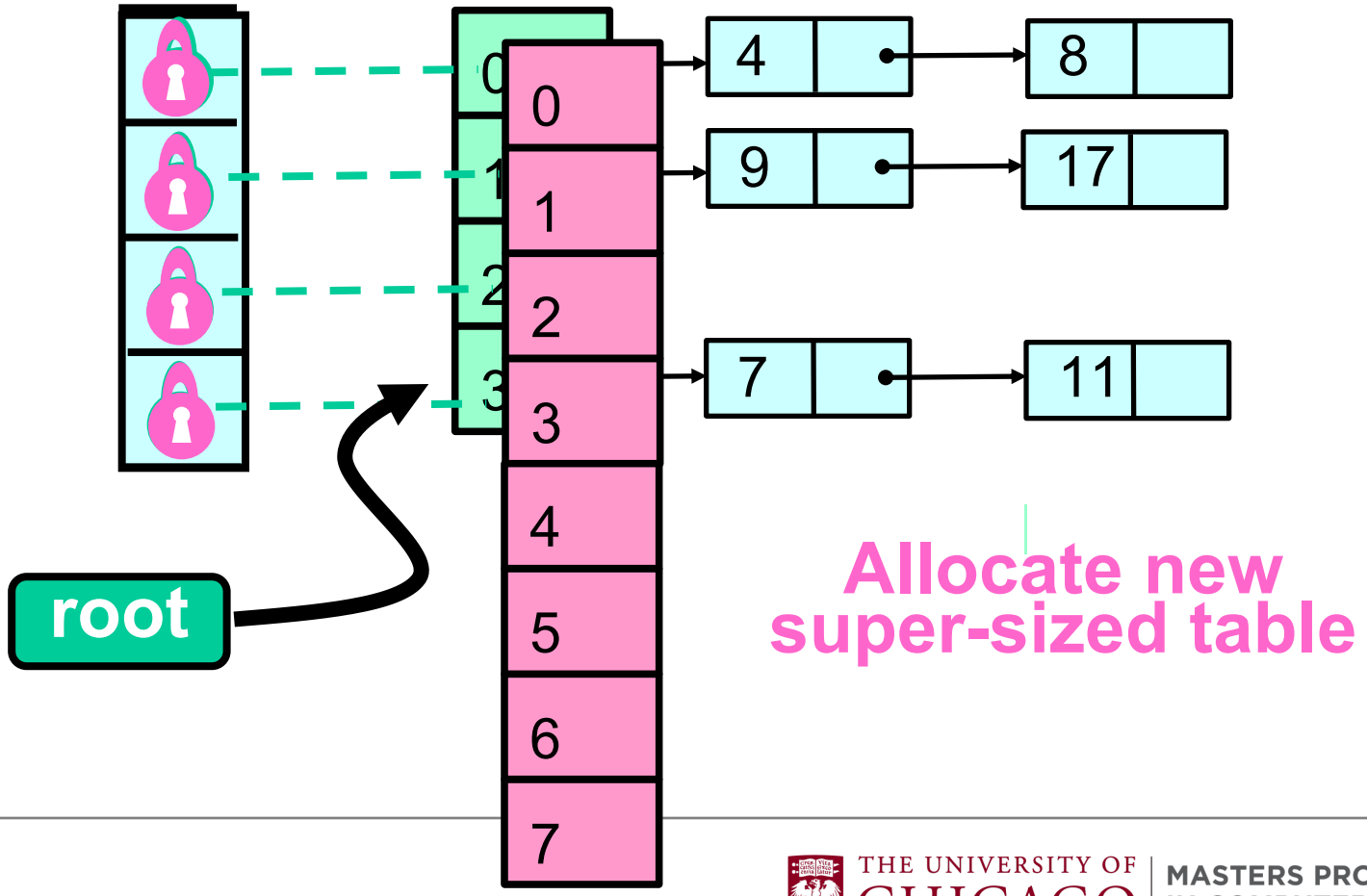


Each lock associated with one bucket

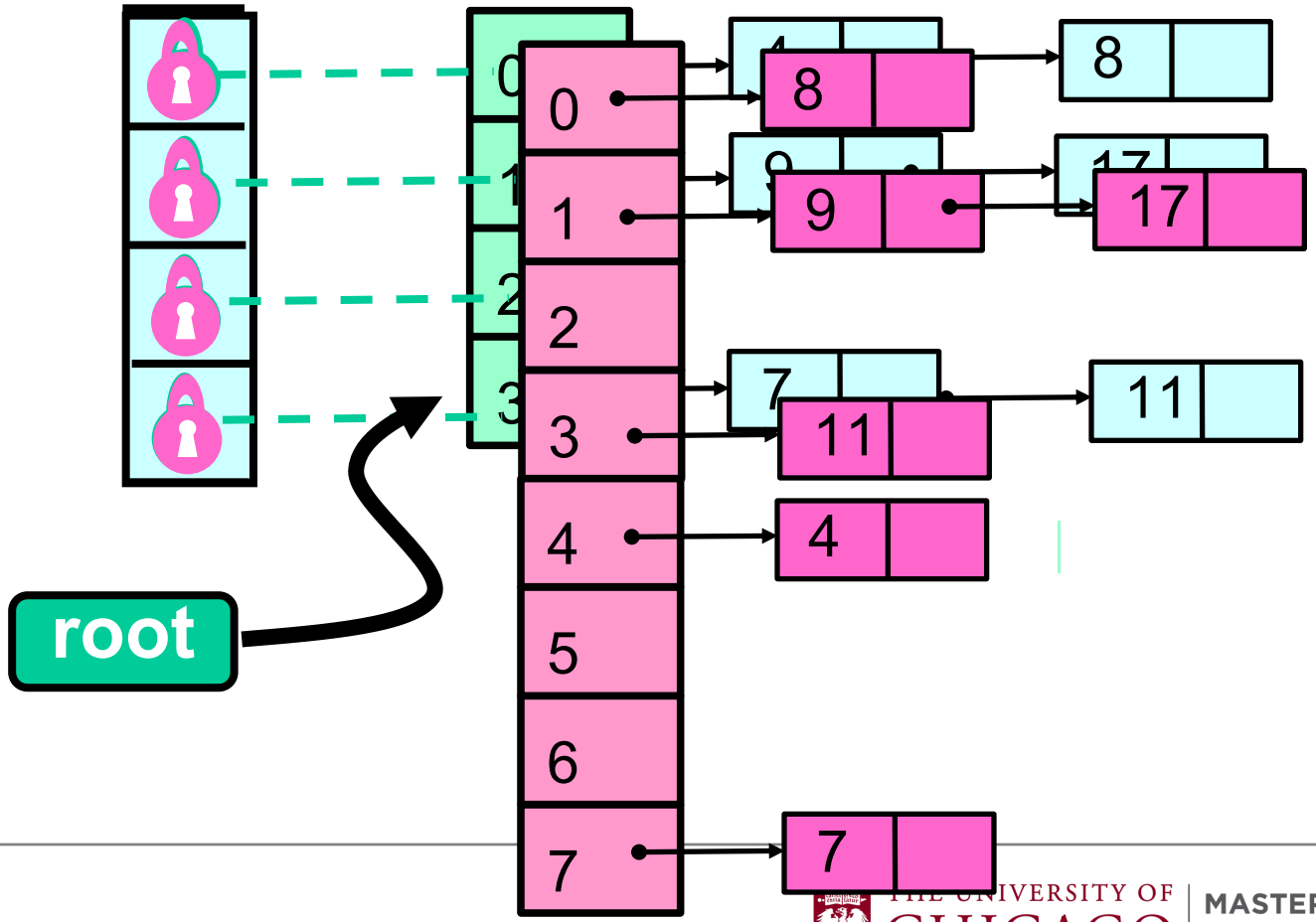
Make sure root reference didn't change between resize decision and lock acquisition



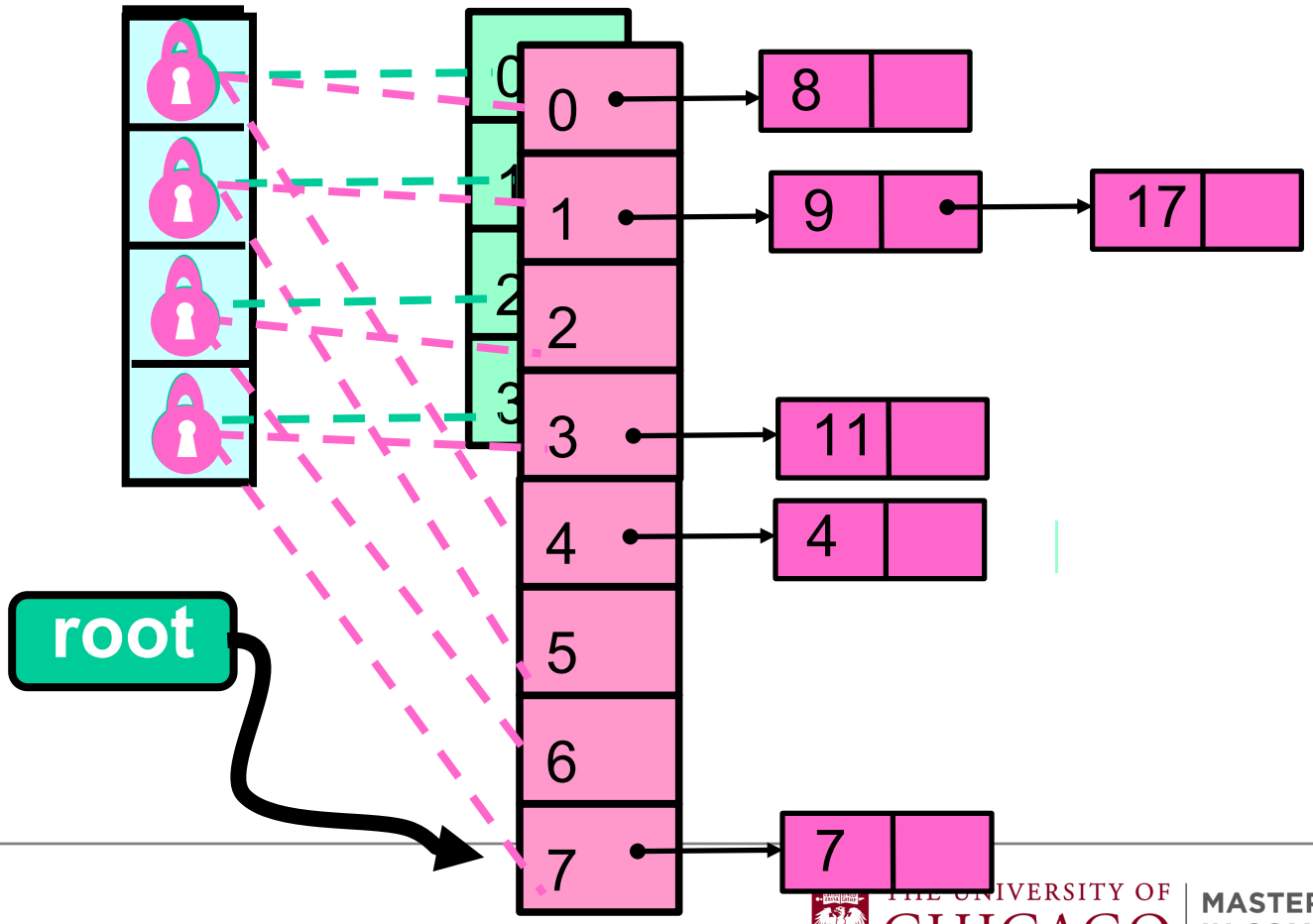
Resize This



Resize This



Striped Locks: each lock now associated with two buckets



Observations

- We grow the table, but not locks
  - Resizing lock array is tricky …
- We use sequential lists
  - Not **LockFreeList** lists
  - If we're locking anyway, why pay?

Fine-Grained Hash Set

```
public class FGHashSet {
 protected RangeLock[] lock;
 protected List[] table;
 public FGHashSet(int capacity) {
  table = new List[capacity];
  lock = new RangeLock[capacity];
  for (int i = 0; i < capacity; i++) {
   lock[i] = new RangeLock();
   table[i] = new LinkedList();
  }} …
```

The add() method

```
public boolean add(Object key) {
 int keyHash
  = key.hashCode() % lock.length;
 synchronized (lock[keyHash]) {
  int tabHash = key.hashCode() %
                    table.length;
  return table[tabHash].add(key);
 }
}
```

Resizing

```
private void resize(int depth,
                    List[] oldTab) {
 synchronized (lock[depth]) {
  if (oldTab == table){
   int next = depth + 1;
   if (next < lock.length)
    resize (next, oldTab);
   else
    sequentialResize();
 }}}
```

Fine-Grained Locking

```
private void resize(int depth,
                    List[] oldTab) {
 synchronized (lock[depth]) {
  if (oldTab == table){
   int next = depth + 1;
   if (next < lock.length)
    resize (next, oldTab);
   else
    sequentialResize();
 }}}
```

resize() calls resize(0,table)

Resizing

```
private void resize(int depth,
                    List[] oldTab) {
 synchronized (lock[depth]) {
  if (oldTab == table){
   int next = depth + 1;
   if (next < lock.length)
    resize (next, oldTab);
   else
    sequentialResize();
 }}}
```

Acquire next lock

Resizing

```
private void resize(int depth,
                    List[] oldTab) {
 synchronized (lock[depth]) {
  if (oldTab == table) {
    int next = depth + 1;
    if (next < lock.length)
     resize (next, oldTab);
    else
     sequentialResize();
  }}}
```

Check that no one else has resized

Resizing

Recursively acquire next lock

```
synchronized (lock[depth]) {
 if (oldTab == table){
  int next = depth + 1;
  if (next < lock.length)
   resize (next, oldTab);
  else
   sequentialResize();
}}}
```

Resizing

Locks acquired, do the work

```
synchronized (lock[depth]) {
 if (oldTab == table){
  int next = depth + 1;
  if (next < lock.length)
   resize (next, oldTab);
  else
   sequentialResize();
}}}
```

Stop The World Resizing

- Resizing stops all concurrent operations
- What about an incremental resize?
- Must avoid locking the table
- A lock-free table + incremental resizing (see textbook)?

Closed (Chained) Hashing

Advantages:

- with N buckets, M items, Uniform h
- retains good performance as table density (M/N) increases → less resizing

Disadvantages:

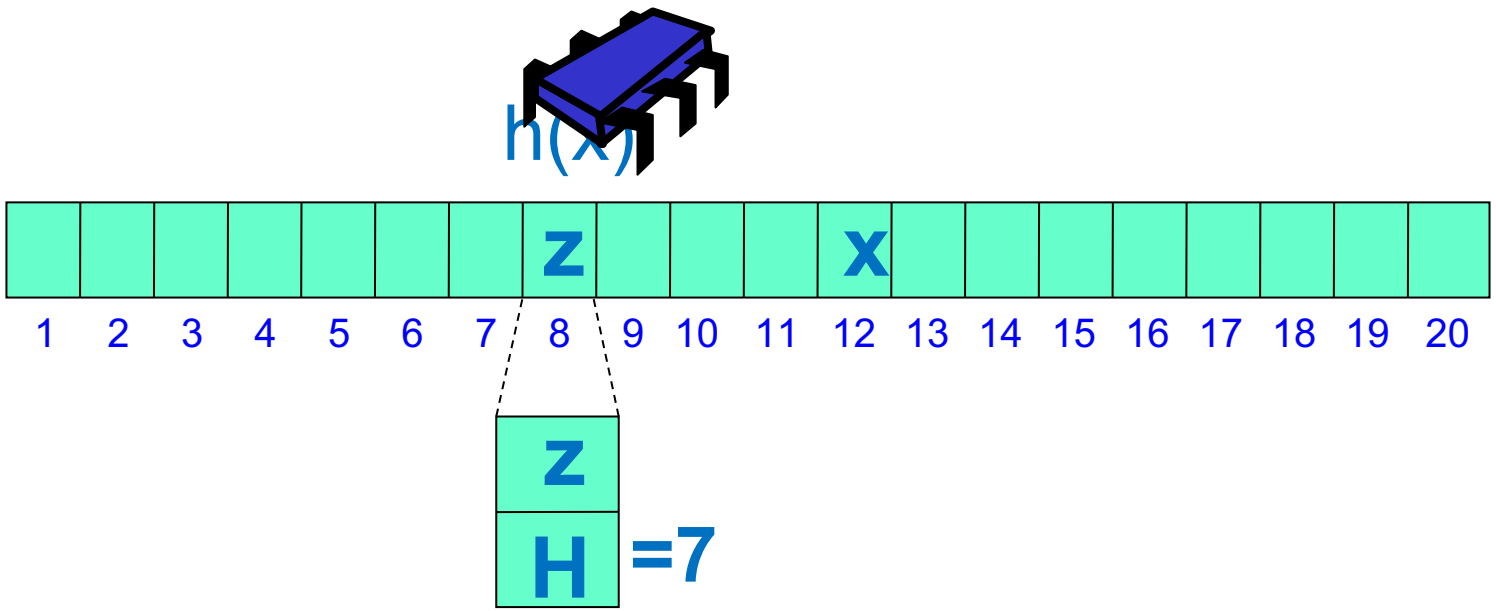
- dynamic memory allocation
- bad cache behavior (no locality)

Oh, did we mention that cache behavior matters on a multicore?

Open Addressed Hashing

Keep all items in an array
One index per bucket
If you have collisions, find an empty bucket and use it
Must know how to find items if they are outside their bucket

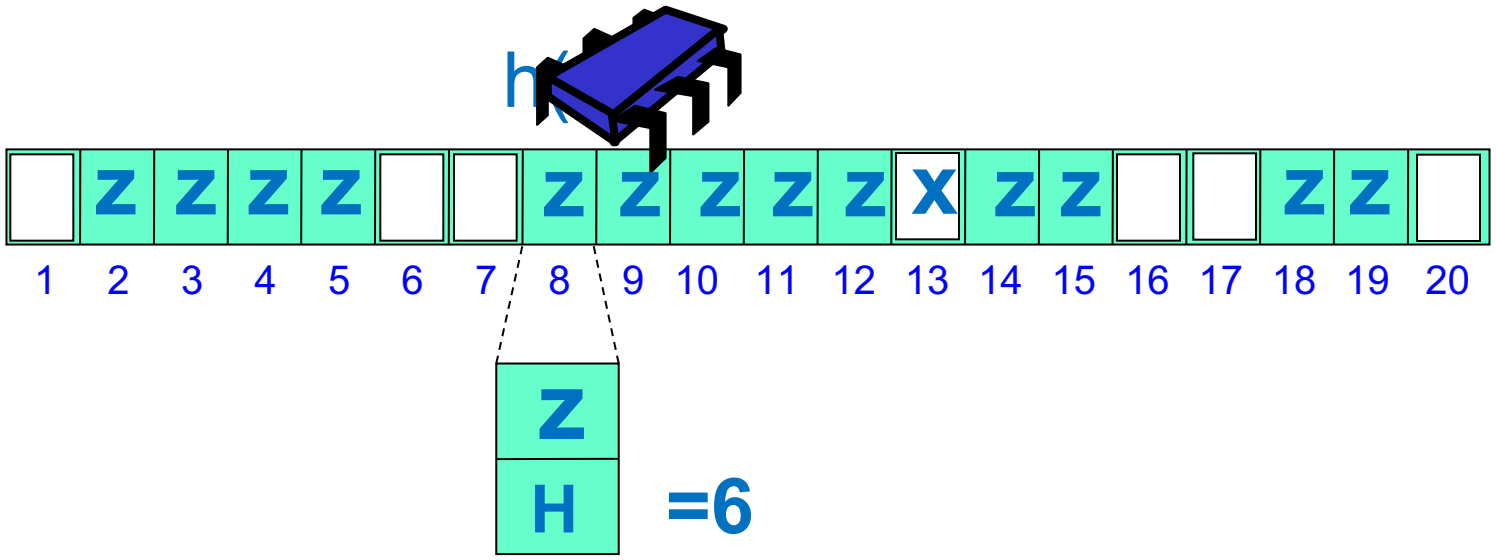
Linear Probing*



`contains(x)` – search linearly from h(x) to h(x) + H recorded in bucket.

*Attributed to Amdahl...

Linear Probing



`add(x)` – put in first empty bucket, and update H.

Expected items in bucket same as Chaining

Expected distance till open slot:

$$\frac{1}{2}\left(1+\left(\frac{1}{1-M/N}\right)^{2}\right)$$

M/N = 0.5 ➔ search 2.5 buckets

M/N = 0.9 ➔ search 50 buckets

Linear Probing

Advantages:

Good locality ➔ fewer cache misses

Disadvantages:

As M/N increases more cache misses

searching 10s of unrelated buckets

"Clustering" of keys into neighboring buckets

Linear Probing

As computation proceeds "Contamination" by deleted items ➔ more cache misses

Concurrent Open Address Hashing

Need to either lock whole chain of displacements (see book)

or have extra space to keep items as they are displaced step by step (Cuckoo hashing, see book).

Summary

Chained hash with striped locking is simple and effective in many cases

See Textbook: *Hopscotch (Concurrent Cuckoo Hashing)* with striped locking great cache behavior

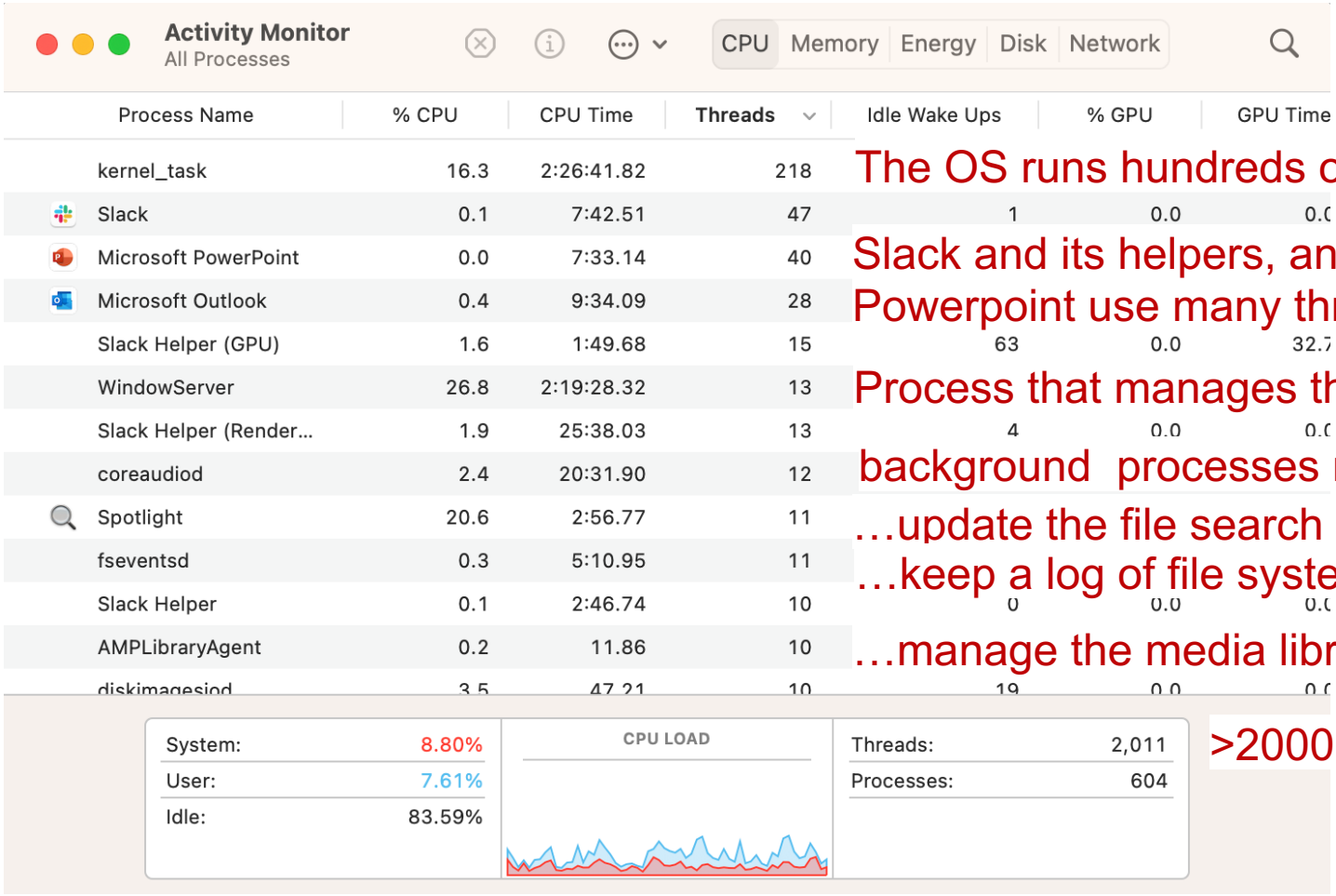
See Textbook: If incremental resizing needed go for *split-ordered*

Thread Scheduling

Why Scheduling?

- We have mostly discussed parallelism to speed up some task
- Instead, we sometimes want “multi-tasking”, not to accelerate one task, but to manage multiple independent tasks
- Examples:
  - Perform work while keeping user-interface responsive
  - Download and install updates in the background
  - Monitor incoming data from multiple sensors while controlling a physical system
- The number of tasks can be much higher than the number of processors (or processor cores)
- Solution: “Scheduling”. A task gets a processor for a while, before another task takes over.
- If the switching is fast enough and every task gets enough time on the processor, this looks like “true” parallelism.

An example from my dual core machine…



The OS runs hundreds of threads

Slack and its helpers, and Powerpoint use many threads here

Process that manages the graphical UI

background processes manage sound

…update the file search index

…keep a log of file system changes

…manage the media library…

>2000 total threads

When do we switch between threads?

We have two basic options.

- Cooperative
  Threads use the processor for as long as they want, and yield it to others when they are done (or when they think it is a good moment to switch)
- Preemptive
  The scheduler takes control away from each thread after some time (based on heuristics).

In either case, we then must decide who gets to run next, e.g. based on priority.

Cooperative, pros/cons

- Pro: Simple. Early operating systems (Mac OS 9 and below, Windows 3.1 and below) worked like this, and embedded systems sometimes still do.
- Pro: Efficient. Processes can yield at suitable times.
- Con: One process can make everyone else starve if e.g.
  - Some computation takes longer than anticipated
  - The process crashes
  - The process waits for I/O while holding the processor
- Con: Difficult to run untrusted code or have multi-user systems with cooperative multitasking

Preemptive, pros/cons

- Pro: Powerful and resilient. Works even if individual programs misbehave.
- Pro: Individual programs do not need to do anything about threading, it is all done by the scheduler.
- Con: Complicated. Threads can be interrupted at arbitrary moments. Compilers/run-time/OS/processor need to deal with this: what happens to register content, cache, program counters, locks held by the process, etc.
- Con: Performance. For example, might interrupt process moments before completion, and pay price for switching back and forth.

Prioritization

All threads should get time slices on a processor, but who should get how much, and in what order?

Considerations:

- Would short delays be observable by the user (e.g. UI, video/audio playback)?
- How much time did the process recently have?
- When is the last time it ran?
- Should users or the process itself have a say in this?

Scheduling/prioritization is a challenging problem. See for example https://kwahome.medium.com/the-first-bug-on-mars-os-scheduling-priority-inversion-and-the-mars-pathfinder-53586a631525

Types of Threads

There are several possible places for implementing threads:

- User-level threads
  - Managed by a thread library or runtime system of the language (e.g., Golang)
  - Typically the OS has no knowledge of these threads.
- Kernel threads
  - Provided and managed by the OS
  - Most OSes support threads at the kernel level.
  - To programs, threads appear to be truly running in parallel
- Hardware threads
  - Provided and managed by the processor
  - For example, “hyperthreading”
  - To the OS, hardware threads appear to be separate processor cores

Hardware threading

- Threads can be managed by hardware
- For example, “hyperthreading”, as previously discussed
- Usually, strict limits on number of threads (e.g. 2 per core)
- Scheduling controlled internally in processor
- Hardware threads may be presented to OS as “logical cores”
- Advantage: Processor may use knowledge of its internal state or the instruction mix of the program to efficiently use resources.

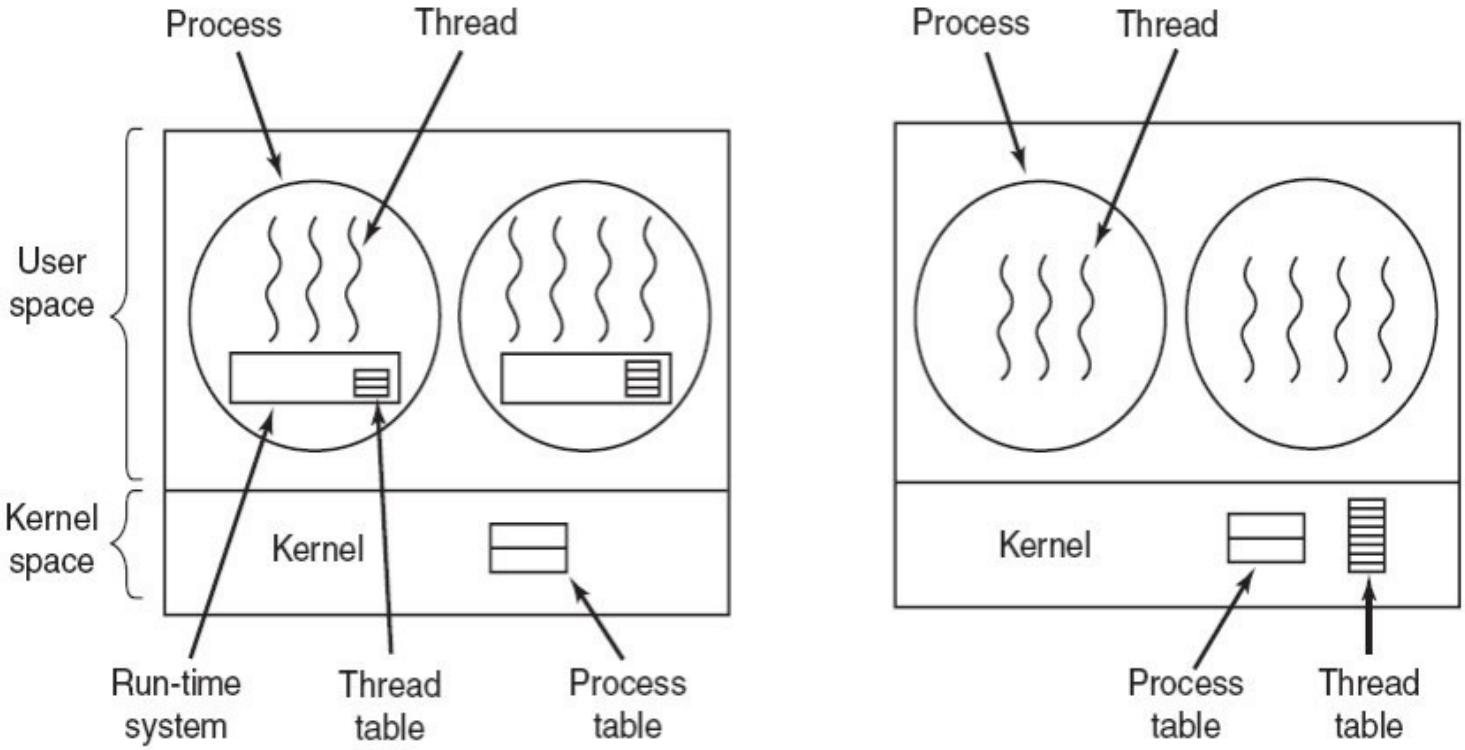
User–level Threads vs. Kernel Threads

Varying degrees of functionality provided by the kernel and user space.

- User-space: System memory allocated to running applications.
- Kernel-space: System memory allocated to the kernel (i.e., the component that controls everything in the OS) and the operating system.

[1]Cite: Tanenbaum, Modern Operating Systems

User–level Threads vs. Kernel Threads



[1]Cite: Tanenbaum, Modern Operating Systems

Pros/Cons: User-level Threads vs. Kernel Threads

- User-level threads
  - Pros: Speed, since the switching of threads can be done without a system call to the operating system (OS), thread creation is fast
  - Cons: If an OS only provides user-level threads then,
    - OS cannot map process threads to multiple CPUs.
    - OS cannot suspend a process if a process thread is performing an I/O operation.
- Kernel threads
  - Pros: The disadvantages of user-level threads can be avoided since the OS knows of their existence and can react appropriately (i.e., schedule another thread if one blocks)
  - Cons: Slow, kernel does creation, scheduling, context-switching, etc.

Multithreading Models

A thread library must map user threads to kernel threads in order to run code on the logical cores.

There are different mappings that exist that an operating system can support, each with its own tradeoffs:

- Many-to-One (N:1): many user threads map to one kernel thread.
- One-to-One (1:1): one user thread to one kernel thread.
- Many-to-Many (N:M): many user threads map to many kernel threads.

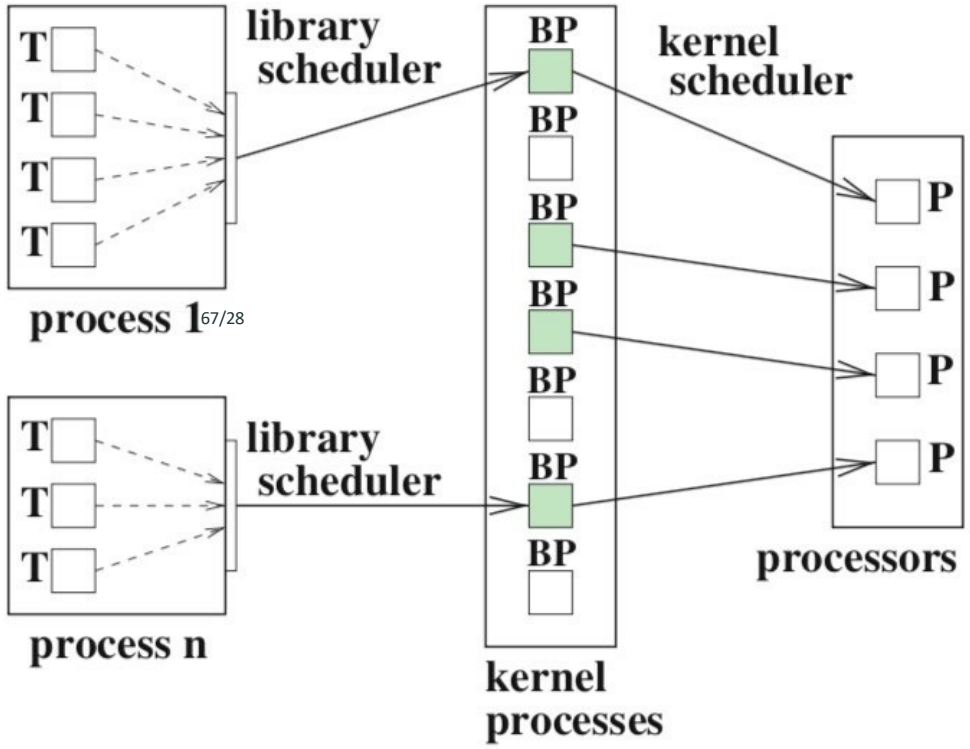
Lets take a look at each.

Many-to-One (N:1)

- This is also known as *user-level threading*
- All user-level threads of a user process are mapped to one kernel thread of the operating system.
- The kernel sees a single process.
- Thread library scheduler determines which user-level thread will be executing for the process (only one thread at a time).
- OS is in charge of mapping a process to a CPU.

N:1 Mapping Illustration

Fig. 3.15 Illustration of a N:1 mapping for thread management without kernel threads. The scheduler of the thread library selects the next thread *T* of the user process for execution. Each user process is assigned to exactly one process *BP* of the operating system. The scheduler of the operating system selects the processes to be executed at a certain time and maps them to the execution resources *P*.



[2]Source: Parallel Programming, Thomas Rauber, Gudula Runger

Pros/Cons of N:1

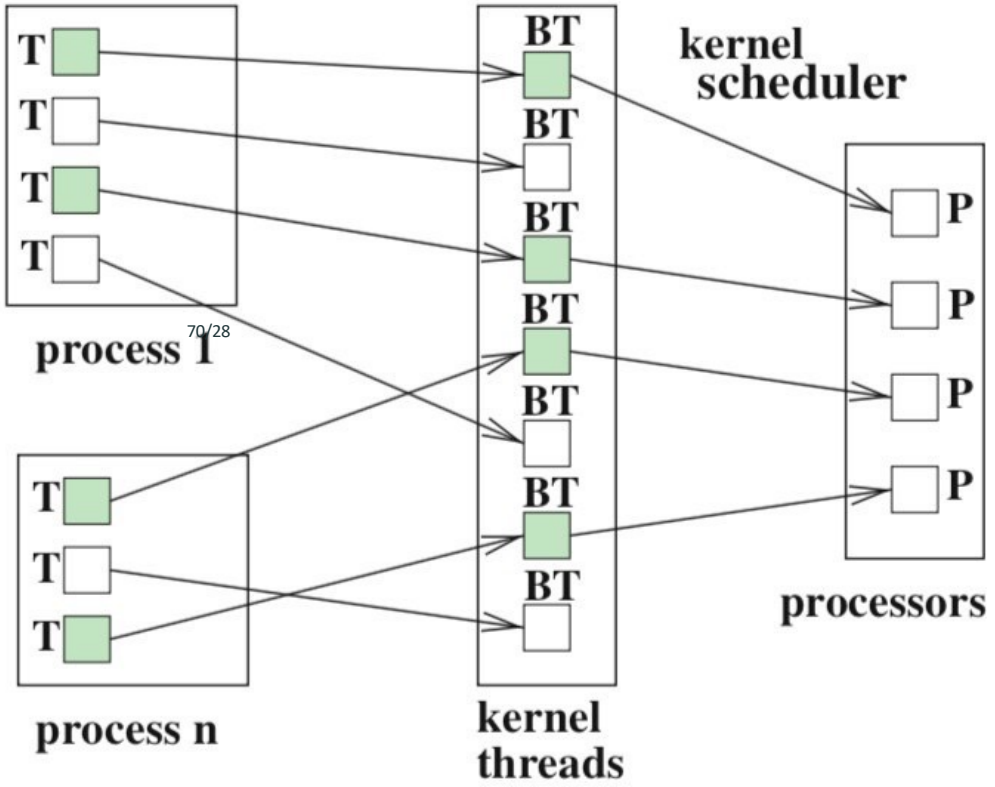
- Pros
  - Creation and context switching is fast because it requires no system calls (i.e., communicating with the kernel, which is expensive).
  - Portability: few system dependencies.
- Cons
  - There is no parallel execution of threads. There's only a single kernel entity backing the N threads; therefore, this model cannot utilize multiple processors.

One-to-One: (1:1)

- This is also known as *kernel-level threading*
- Generates a kernel thread for each user-level thread.
- The scheduler of the operating system selects which kernel threads are executed at which point in time.
- OS is in charge of mapping threads to a CPU(s)
- No need for a library scheduler since each user-level thread is assigned to exactly one kernel thread.

1:1 Mapping Illustration

Fig. 3.16 Illustration of a 1:1 mapping for thread management with kernel threads. Each user-level thread T is assigned to one kernel thread BT. The kernel threads BT are mapped to execution resources P by the scheduler of the operating system.



70/28

3

[3]Source: Parallel Programming, Thomas Rauber, Gudula Runger

Pros/Cons of 1:1

- Pros
  - Allows for more concurrency. When one thread blocks, other threads can be scheduled.
- Cons
  - Depending on the operating system this model can be expensive when it comes to creation, context switching, and deletion of threads.
  - All thread operations involve the kernel

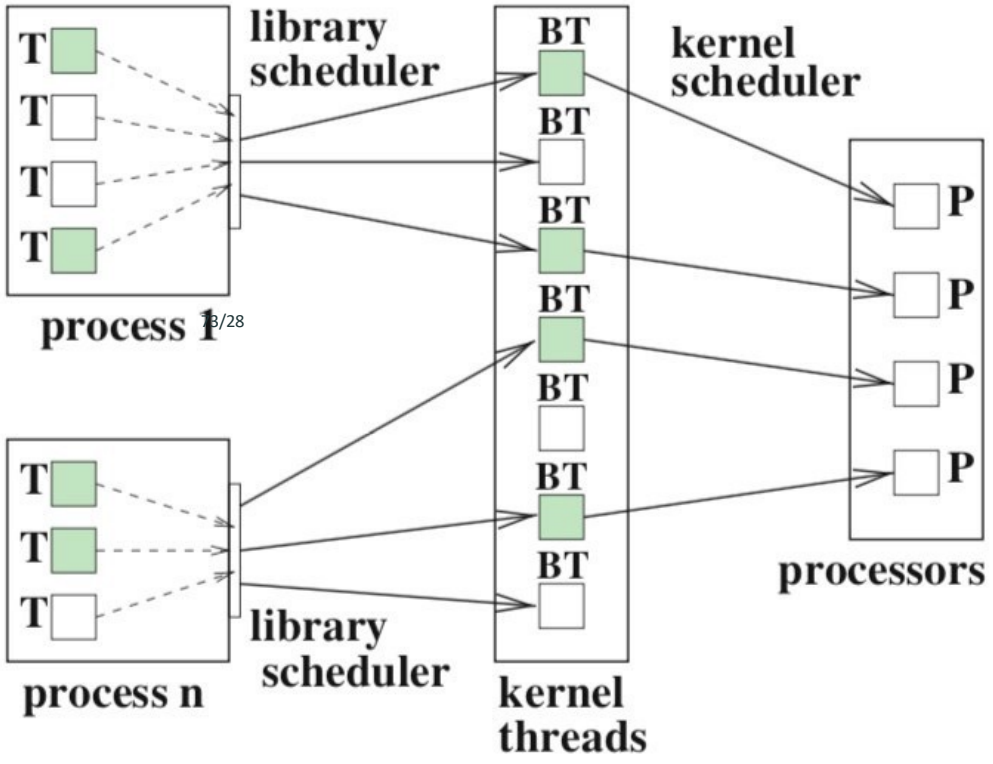
Many-to-Many: N:M

N = User threads, M = Kernel Threads

- This is also known as *Hybrid threading*
- Two-level scheduling where the thread library scheduler assigns user-level threads to a given set of kernel threads.
- At any point in time, a user threads can be mapped to a different kernel thread (no fixed mapping)
- N is greater than M.
- Correspondingly, a kernel thread can execute different user threads at any point in time.
- The Go runtime uses this mapping to manage goroutines.

N:M Mapping Illustration

Fig. 3.17 Illustration of a N:M mapping for thread management with kernel threads using a two-level scheduling. User-level threads T of different processes are assigned to a set of kernel threads BT (N:M mapping) which are then mapped by the scheduler of the operating system to execution resources P.



[4]Source: Parallel Programming, Thomas Rauber, Gudula Runger

Pros/Cons of N:M

- Pros
  - This model is flexible because the OS creates kernel threads for physical concurrency and the Applications creates user threads for application concurrency.
- Cons
  - It's quite a complex model to implement. Most systems use 1:1 mapping.

Coroutines

Goroutines are not technically threads, but really based off the concept of a coroutine:

- A unit of execution even lighter in weight than a thread.
- They are like user-level threads but have little to no user-space support for their scheduling and execution.
- They are cooperatively scheduled, requiring an explicit yield to move to another coroutine (e.g., runtime schedule call, i/o call, etc.)
- They enable differing programming paradigms and I/O models such as CSP, which we will talk about next.

Goroutines as Coroutines

Having Goroutines be coroutines have a few benefits over using kernel threads:

- Memory consumption: Threads require O(1MB) of memory vs O(1Kb) memory for a coroutine.
- Switch Cost: When a context switch occurs, threads have to save a large amount of state information (general purpose registers, PC (Program Counter), SP (Stack Pointer), segment registers etc), whereas gorountines usually have to save just the program counter and stack pointer, and a few registers.

Scheduling of Goroutines

Since goroutines are scheduled *cooperatively*, goroutines yield the control periodically when they are idle or logically blocked in order to run other Goroutines concurrently. This switch is is done at well defined points:

- Channels send and receive operations, if those operations would block.
- The Go statement, although there is no guarantee that the new Goroutine will be scheduled immediately.
- Blocking syscalls like file and network operations.
- After being stopped for a garbage collection cycle.

Scheduling of Goroutines Internally

Go uses three entities to explain the scheduler,

- Processor (P)
- Kernel Thread (M)
- Goroutines (G)

Per Go application, the number of threads available for Goroutines to run is equal to the *GOMAXPROCS*:

- `func GOMAXPROCS(n int) int`: sets the maximum number of CPUs that can be executing simultaneously and returns the previous setting
- If $n < 1$, it does not change the current setting
- Defaults to the number of logical cores for the system (i.e., `func NumCPU() int`)

Scheduling of Goroutines Internally (cont.)

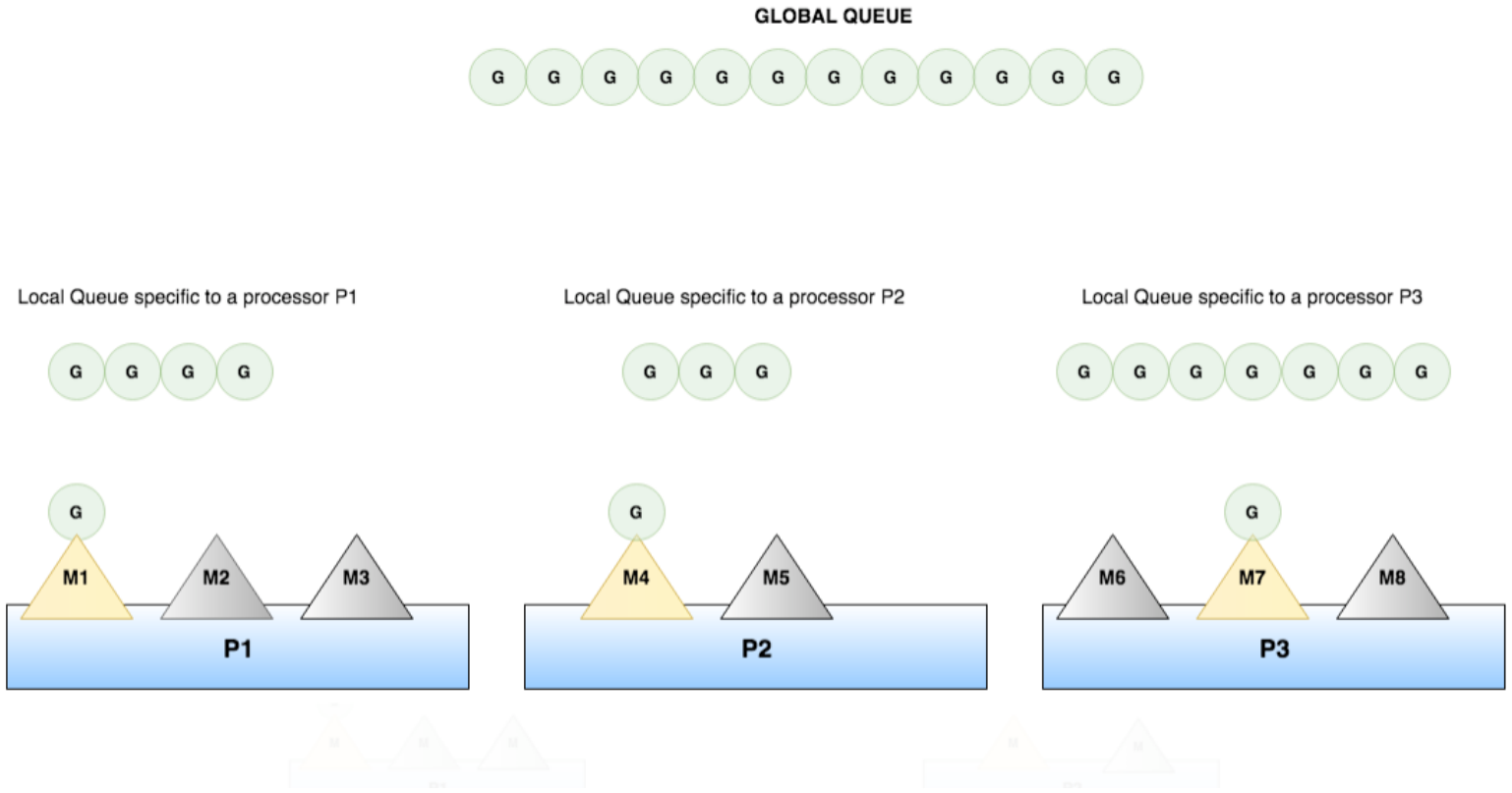
Golang uses an *M* : *N* scheduler which means that *M* goroutines need to be scheduled on *N* Kernel threads that runs at most GOMAXPROCS number of processors (*N* < GOMAXPROCS)

How it works:

"Every P has a local Goroutine queue. There's also a global Goroutine queue which contains runnable Goroutines. Each M should be assigned to a P. Ps may have no Ms if they are blocked or in a system call. At any time, there are at most GOMAXPROCS number of P and only one M can run per P. More Ms can be created by the scheduler if required." [6]

[6]Cite: https://riteeksrivastava.medium.com/a-complete-journey-with-goroutines-8472630c7f5c

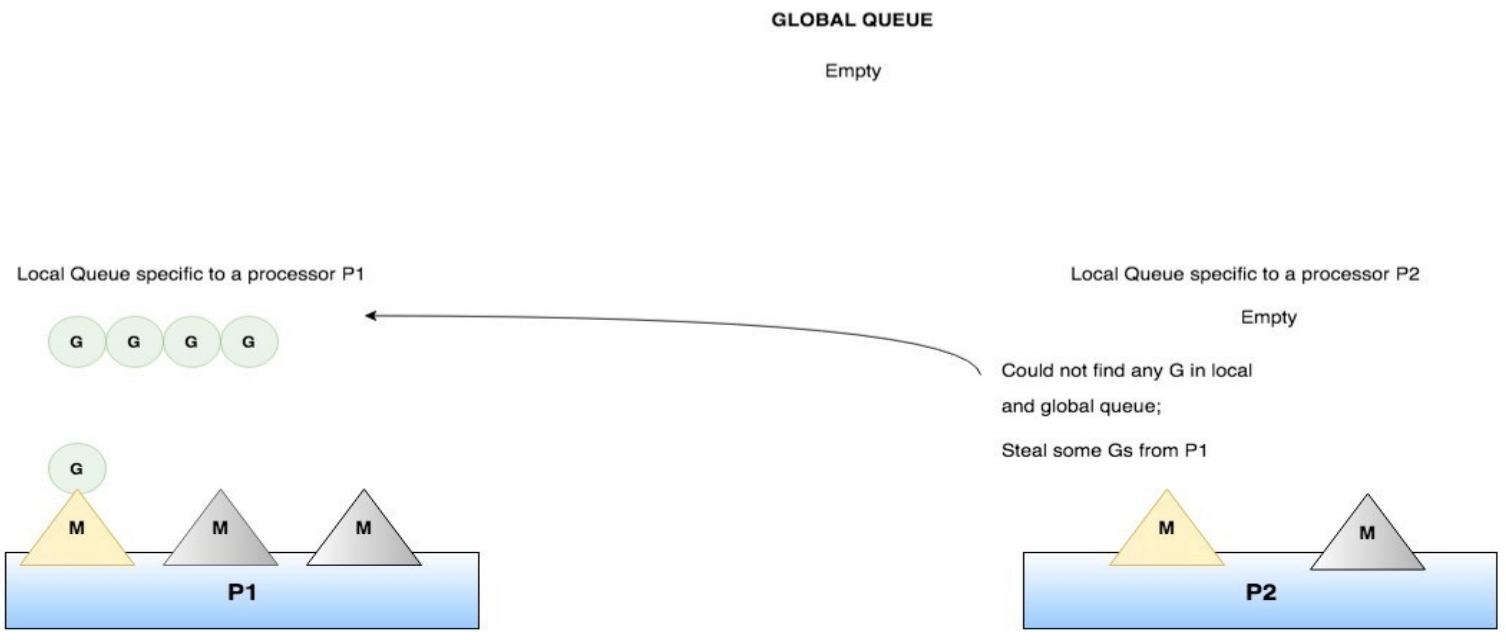
Scheduling of Goroutines Internally (cont.)



Cite: https://riteeksrivastava.medium.com/a-complete-journey-with-goroutines-8472630c7f5c

Scheduling of Goroutines Internally (cont.)

Each round of scheduling, the scheduler finds a runnable Goroutine and executes. If a Processor's queue is empty then it will try to *steal* Goroutines from other processor local queues.



Cite: https://riteeksrivastava.medium.com/a-complete-journey-with-goroutines-8472630c7f5c

Go's Concurrency Model

The preferred concurrency model when it comes to sharing resources between goroutines is "Do not communicate by sharing memory; instead, share memory by communicating".

To do this, Go uses a powerful notion of channels:

- A channel in Go provides a connection between two goroutines, allowing them to communicate.

```
// Declaring and initializing.
var c chan int
c = make(chan int)
// or
c := make(chan int)
```

Channel Examples

Demos:

- simple.go
- channel-buffering.go
- channel-directions.go
- channel-synchronization.go

Generator

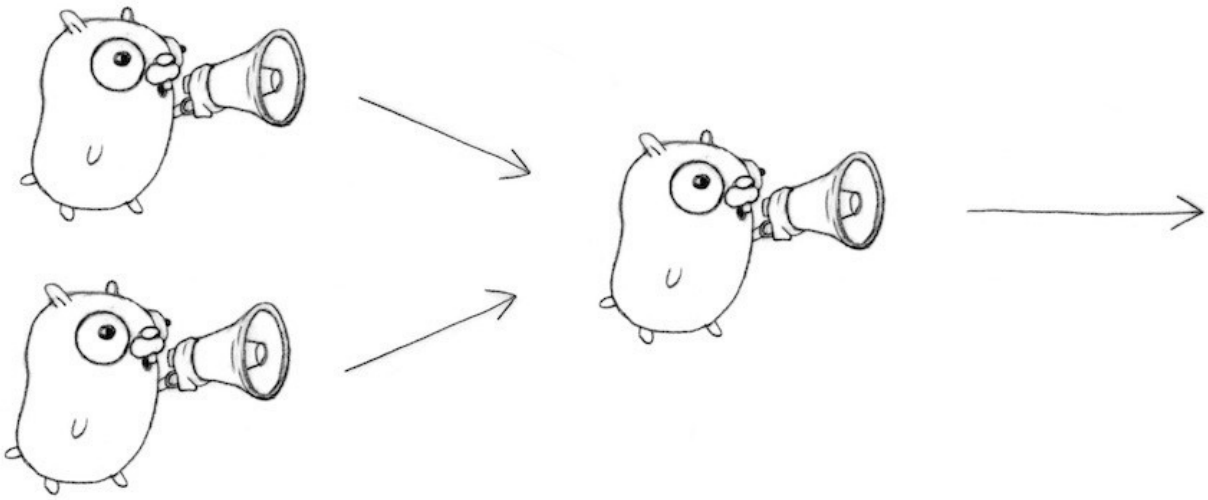
A generator is function that returns a channel

- As with other primitive types (integers, strings, etc.), channels are also first class values.
- Generators allow us to have more instances of a service.

Demo: See example in generator.go

Multiplexing

These programs make Bob and Sally count in lockstep. We can instead use a fan-in function to let whosoever is ready talk.



Demo: See example in m6/discussion/fanin/fanin.go

Select

The select statement lets you wait on multiple channel operations.

It's like a switch, but each case is a communication:

- When a select statement is reached, all cases are evaluated.
- The select blocks until one communication can proceed, which then performs the code inside its case block
- If multiple channels receive a value then select chooses pseudo-randomly a case to execute.
- A default clause, if present, executes immediately if no channel is ready.

```
select {
    case v1 := <-c1:
        fmt.Printf("received %v from c1\n", v1)
    case c3 <- 23:
        fmt.Printf("sent %v to c3\n", 23)
    default:
        fmt.Printf("no one was ready to communicate\n") }
}
```

Completion and Timeout

Timeouts and the closing of a channels are two ways to signal to a goroutine that a task has completed successfully or time has ran out:

- Timeouts: timeouts.go
- Ranging over, and closing channels: range-channel.go

Preventing goroutine leaks

```
func main() {
        newRandStream := func() <-chan int {
                randStream := make(chan int)
                go func() {
                        defer fmt.Println("newRandStream closure exited.")
                        defer close(randStream)
                        for {
                                randStream <- rand.Int()
                        }
                }()

                return randStream
        }

        randStream := newRandStream()
        fmt.Println("3 random ints:")
        for i := 1; i <= 3; i++ {
                fmt.Printf("%d: %d\n", i, <-randStream)
        }
}
```

Preventing goroutine leaks

```
func main() {
        newRandStream := func() <-chan int {
                randStream := make(chan int)
                go func() {
                        defer fmt.Println("newRandStream closure exited.")
                        defer close(randStream)
                        for {
                                randStream <- rand.Int()
                        }
                }()

                return randStream
        }

        randStream := newRandStream()
        fmt.Println("3 random ints:")
        for i := 1; i <= 3; i++ {
                fmt.Printf("%d: %d\n", i, <-randStream)
        }
}
```

```go
func main() {
	newRandStream := func(done <-chan interface{}, wg *sync.WaitGroup) <-chan int {
		randStream := make(chan int)
		go func() {
			defer wg.Done()
			defer fmt.Println("newRandStream closure exited.")
			defer close(randStream)
			for {
				select {
				case randStream <- rand.Int():
				case <-done:
					return
				}
			}
		}()

		return randStream
	}
	var wg sync.WaitGroup
	wg.Add(1)
	done := make(chan interface{})
	randStream := newRandStream(done, &wg)
	fmt.Println("3 random ints:")
	for i := 1; i <= 3; i++ {
		fmt.Printf("%d: %d\n", i, <-randStream)
	}
	close(done)

	// Simulate ongoing work
	wg.Wait()
}
```

Next week

- Detailed implementation of work-stealing
- Parallel programming patterns:
  - Fan-out fan-in
  - Map-reduce
  - Pipelining
  - BSP